

GuardianX 개발 작업지시서 v3.0

대상: 개발팀 / AI 코딩 에이전트 · 기준 소스: C:\GuardianX\guardianx-source (실측 판독 완료) · 작성일: 2026-08-11 상위 문서: 전략보고서 v1.0 / PRD v1.0 / 소스실측분석 v1.0 / Value 설계서 v1.0

v3.0 변경점 — 실제 실행(2026-08-13, W0-1 STOP) 결과를 반영한 개정 1. 사전 결정 레지스트리 신설 (agent/decisions.yaml , 20건) — 에이전트가 CLARIFY로 멈추기 전에 조회한다 2. W0 재편 — W0-0 (저장소 초기화) 신설, W0-1 을 W0-1a (에이전트)/ W0-1b (사람)로 분할, W0-5 · W0-6 추가 → 58개 티켓 3. gate 이원화 — human-required 9건만 STOP, evidence-review 49건은 증거 제출로 자동 통과 → 정지 지점 22 → 9 4. Phase 자동 승격 — R1→R2→R3를 조건 충족 시 에이전트가 스스로 넘어간다 5. 정본 무결성 검사 — PDF 역복원으로 7건 누락된 사고 방지 (manifest + SHA256)

실행 중 발견되어 정정된 것 (에이전트 REVIEW의 성과)

발견	원래 티켓의 오류	정정
저장소에 커밋이 하나도 없음	최초 커밋 시 실키 15개가 히스토리에 박힘	W0-0 신설, 치환 후 커밋으로 순서 고정
CI가 존재하지 않음	DoD "CI 잡 통과"를 재현 불가	로컬 pre-commit을 실패 차단선으로, CI는 선제 배치
GCS_APIKEY = VITE_CGS_APIKEY 동일 UUID	재발급해도 같은 상태 반복	값 분리 + 도메인 제한 + W0-5 proxy 전환
baseline 부재로 기존 45개 앱이 통째로 위반 판정	컨벤션 검사가 무력화	W0-0 커밋을 baseline으로 명시, 없으면 SKIP
yarn 미설치 / celery 미설치	환경 문제를 코드 실패로 오인	npm 확정, 백엔드는 컨테이너, env-unavailable 분리

동봉 파일 (agent/)

파일	용도	누가 쓰나
tickets.yaml	58개 티켓 정보 (manifest + SHA256)	에이전트가 status/evidence만 수정
decisions.yaml	사전 결정 20건 — CLARIFY 억제용	에이전트 읽기 전용, 사람이 확정
AGENT_LOOP.md	루프 실행 프롬프트 v2.0 — 통째로 붙여넣는다	사람이 1회 투입
verify_ticket.sh	공통 게이트 자동화	에이전트·CI
tickets.sha256	정본 체크섬	무결성 검사
본 문서	규칙·컨벤션·금지구역	에이전트 읽기 전용 / 사람 리뷰

⚠ tickets.yaml 을 PDF나 이 문서에서 재구성하지 마십시오. v2.0 실행 시 실제로 PDF 역복원을 시도해 R2 4건·R3 3건이 누락됐습니다. 문서는 사람이 읽는 요약이고, 기계 판독 정보는 agent/tickets.yaml 하나뿐입니다.

0. 이 지시서를 읽는 법 (에이전트/개발자 공통 규칙)

0.1 최우선 원칙 — 5개

1. **재사용 > 신규.** 이 코드베이스는 이미 43개 라우트·45개 Django 앱을 가지고 있다. 새 모듈을 만들기 전에 **반드시 기존 앱을 먼저 찾는다.**
2. **수직 슬라이스로 완료한다.** 모델→API→화면→PDF까지 하나를 끝내고 다음으로 간다. 레이어별 일괄 작업 금지.
3. **완료 판정은 코드가 아니라 화면이다.** 각 티켓의 DoD 는 "사람이 이 화면에서 이 버튼을 누르면 이것이 나온다"로 쓰여 있다. 그것을 재현하지 못하면 미완료다.
4. **리팩터링 금지 구역을 지킨다.** (§ 0.4) 눈에 거슬려도 지금은 건드리지 않는다.
5. **매뉴얼과 코드를 동시에 갱신한다.** GS인증은 매뉴얼 기재 기능과 실제 동작의 불일치를 결함으로 처리한다. 기능 머지 시 사용자매뉴얼 항목을 같은 PR에 포함한다.

0.2 전 Phase 로드맵 (54개 티켓)

Phase	기간	티켓	목표	상태
R1 "Certify"	2026.09~12 (코드 동결)	28개 (W0-0~W6-2)	인증 통과 빌드 + 첫 유료 레퍼런스 판매 가능	ready
R2 "Station"	2027.2Q	14개 (W6-1~W11-2)	무인 스테이션 구독 + GPU 파이프라인 전환 + 폐쇄망/클라우드 판	backlog
R3 "Mission"	2027.4Q	8개 (W12~W16)	임무 API·에이전틱·증거체인·파일럿 앱	backlog
BACKLOG	조건부 승격	8개 (B-01~B-08)	표준 예비 후보. promote_when 충족 시 사람이 phase 부여	backlog
합계	—	58개	human-required 9 · evidence-review 49	—

R1 — W0-0 저장소 초기화 (최선행, D-004)
└ W0-1a 시크릿 게이트 → W0-1b 실키 재발급(사람) → W0-5 proxy 전환 → W0-6 원격CI(사람)
— W0-2/3/4 → W1 자동보고서 (선행 필수) —
└ W2 이벤트센터 — W3 3계층 대시보드
└ W4 라이선스 미터링
└ W5 VMS 게이트웨이 [독립·병렬]
└ W6-0 H100 실측 [독립·병렬]
└ W6-2 인터페이스 계약 고정 [R2 대비 선제]

R2 — W6-1 GPU 파이프라인 — W9 SAHI/열화상/점검
└ W7 DJI Dock → W8 무인 스테이션
└ W10 폐쇄망 → W11 CSAP·K8s/MIG

R3 — W12 임무API · W13 에이전틱 · W14 증거체인 · W15 변화탐지 · W16 파일럿앱

BACKLOG — B-01 안티드론 · B-02 벤치마크 · B-03 지도통합 · B-04 UI통일
B-05 배송 라이선스분리 · B-06 다국어 · B-07 AI자산화 · B-08 상표

W0는 다른 모든 작업의 선행조건이다. W1·W2·W4·W5·W6-0은 병렬 가능.

⚠ B-05(배송 라이선스 분리)와 B-07(AI 자산화)·B-08(상표)은 **promote_when: 즉시 다**. 특히 B-07은 개발이 아니라 **경영 실사 사안**이며, 외부 AI 서비스 3종의 모델·데이터 소유권이 이 사업 해자의 실체를 결정한다. 개발 착수와 무관하게 지금 확인해야 한다.

0.3 티켓 표기

- [P0] R1 필수 / [P1] R1 목표 / [P2] R2 이후
- 공수 : S(≤2일) / M(3~5일) / L(1~2주) / XL(2주+)
- 재사용 : 기존 코드에 붙이는 작업 / 신규 : 새로 만드는 작업

0.4 리팩터링 금지 구역 (R1 동안 건드리지 말 것)

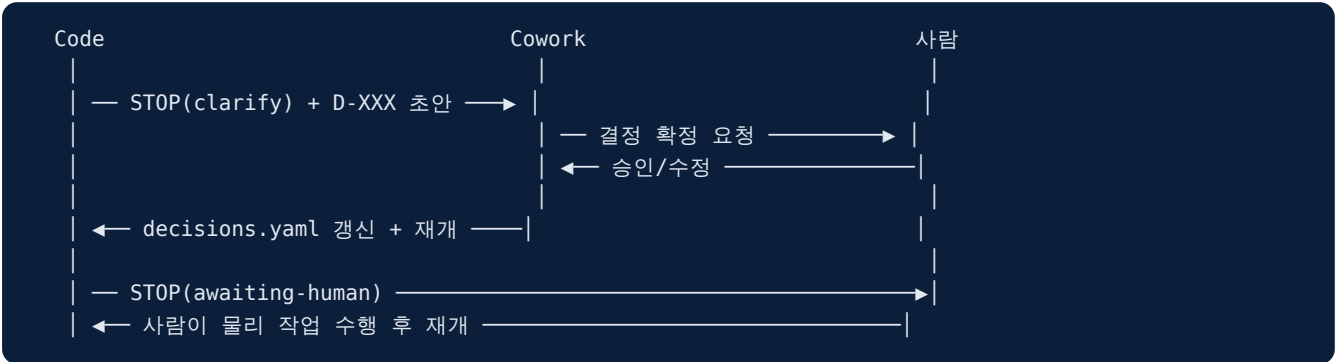
대상	이유
delivery / orders / terminals 3개 앱 (269 파일)	배송 라인. 라이선스 플래그로 숨기기만 하고 코드는 그대로 둔다
rw-core / dj-core (사내 private 패키지)	프레임워크. 여기 손대면 전 화면이 흔들린다
지도 3종 통합 (MapForRoute / *Google / *Unified)	P1. R1 범위 아님. *Unified 래퍼만 신규 화면에서 사용
UI 라이브러리 통일 (MUI/AntD/Bootstrap)	P1. 신규 화면만 AntD로 통일해서 만든다. 기존 화면 개종 금지
FormRoute.tsx (69KB)	이미 동작하는 경로 편집기. 리팩터링 유혹 차단

0.45 Cowork ↔ Code 협업 프로토콜

이 프로젝트는 두 개의 AI가 역할을 나눠 돌아간다. 역할이 섞이면 둘 다 잘못한다.

	Cowork (기획·지시)	Code (구현)
하는 일	티켓 작성·수정, 결정(D-xxx) 확정, Phase 정의, 문서 산출	코드 작성, 테스트, 검증, 증거 수집
안 하는 일	코드 직접 수정	티켓 spec·dod 변경, decisions.yaml 수정
쓰는 파일	tickets.yaml (spec/dod), decisions.yaml	tickets.yaml (status/evidence), 소스

왕복 사이클 (STOP → 결정 → 재개)



왕복을 최소화하는 3가지 규칙

1. Code는 CLARIFY 시 반드시 D-XXX 초안 을 제안한다. options 와 recommend 를 포함해서 → 사람은 "A로 해"만 답하면 되고, Cowork는 그것을 그대로 decisions.yaml 에 넣는다.
2. Cowork는 결정을 확정할 때 dod_override 까지 함께 쓴다. 결정만 주고 DoD가 그대로면 Code가 같은 자리에서 또 멈춘다. (D-001이 그 예)
3. 한 번 멈출 때 여러 결정을 몰아서 답한다. Code는 STOP 시점에 "앞으로 나올 것으로 예상되는 결정"도 함께 초안으로 제안한다.

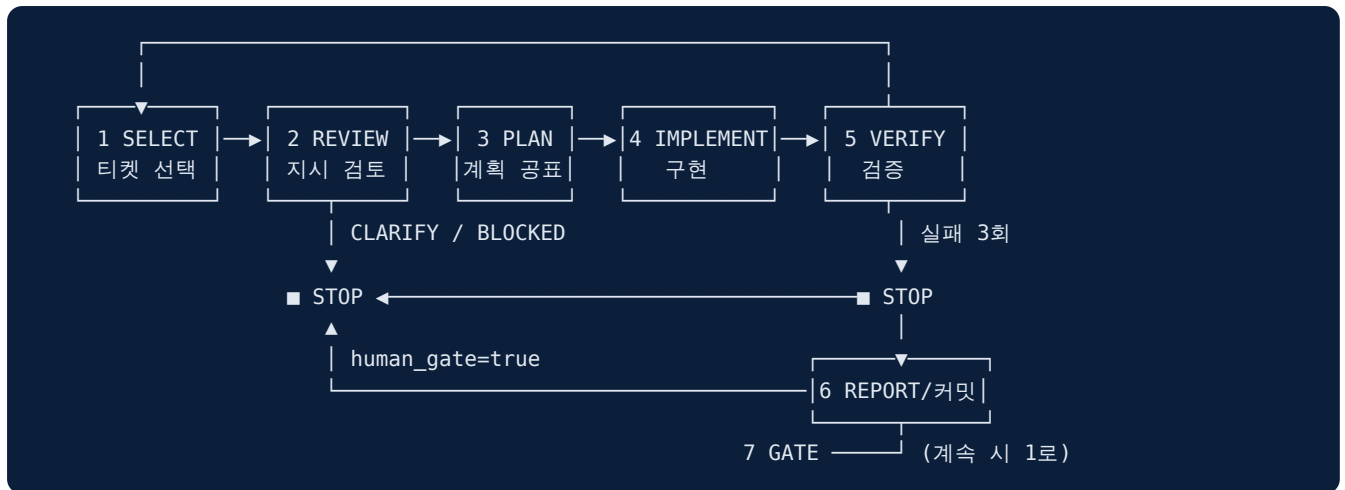
이 프로토콜이 실제로 작동한 사례 (2026-08-13)

Code가 W0-1에서 STOP하며 4개 질문을 제기했다 → Cowork가 D-001~D-007로 확정하고 W0-0 티켓 신설·W0-1 분할·DoD 재정의까지 반영했다. Code가 발견한 "커밋이 없어 최초 커밋 시 실키가 히스토리에 박힌다"는 티켓 설계자(Cowork)의 오류였고,

REVIEW 단계가 그것을 잡아냈다. → 이것이 REVIEW를 강제한 이유다. 에이전트는 지시를 검증하는 역할도 한다.

0.5 🔄 자동 구현 루프 (에이전트 운영 규칙)

이 지시서는 사람이 읽는 문서인 동시에 에이전트가 실행하는 명세다. `agent/AGENT_LOOP.md` 를 코딩 에이전트에 투입하면 아래 사이클이 자동으로 돈다.



이 루프의 핵심은 STEP 2 REVIEW다

에이전트가 티켓을 그대로 믿고 구현하지 않게 만드는 단계다. 5개를 강제한다.

#	점검	실패 시
2-1	<code>reuse_targets</code> 파일을 전부 실제로 읽었는가	진행 불가
2-2	티켓의 가정 이 실제 코드와 일치 하는가 (모델·함수·라우트가 있는가)	CLARIFY → STOP
2-3	수정 대상이 § 0.4 금지구역에 속하는가	BLOCKED → STOP
2-4	<code>dod</code> 재현에 필요한 정보가 티켓에 다 있는가	CLARIFY → STOP
2-5	다른 티켓·기존 기능을 깨뜨릴 가능성은	위험 목록 출력

왜 이게 필요한가: 이 코드베이스는 이미 43개 라우트·45개 앱을 가지고 있다. 에이전트가 기존 구현을 못 보고 새로 만드는 순간 중복 구현 + 데이터 이원화가 발생하고, 그 비용은 처음부터 사람이 짰 것보다 크다. REVIEW는 그 사고를 막는 유일한 장치다.

완료 판정은 코드가 아니라 DoD 재현

STEP 5에서 DoD를 실제로 재현하고 증거(스크린샷·로그·측정값)를 `tickets.yaml` 의 `evidence` 에 남긴다. "코드는 다 짰다"는 완료가 아니다.

정지 조건 (9종) — 멈추는 것은 정상 동작이다

`no-ready-ticket` / `clarify` / `blocked` / `verify-failed` / `awaiting-human-review` / `phase-complete` / `iteration-limit` / `destructive-migration` / `secret-touch`

절대 금지 (하나라도 어기면 해당 작업 무효)

- `reuse_targets` 를 읽지 않고 구현
- DoD 미재현 상태로 `done` 처리
- 티켓에 없는 리팩터링 끼워 넣기
- 테넌트 격리 테스트 비활성화·스킵·수정 (테스트가 틀린 것 같으면 STOP하고 보고)
- `.env*` 에 실제 값 기입
- 권한 우회 목록에 항목 추가 (W0-2가 정확히 그 실수의 결과다)

7. CLARIFY / BLOCKED 판정을 내려놓고 그냥 진행

사람만 할 수 있는 일 (에이전트가 대신하지 않는다)

W0-1 실키 재발급 · W5-3 이노덱 연동 검증 · W6-0 H100 임대 실측 · W11-1 CSAP 신청 · B-07 AI 소유권 실사 · B-08 상표 출원 · Phase 승격 결정 · `human_gate: true` 티켓 최종 승인

실행 방법

```
# 1회 세팅
cp -r agent/ <repo>/docs/agent/

# 에이전트에 투입 (Claude Code 예시)
claude "docs/agent/AGENT_L00P.md 를 읽고 그대로 실행하라. ACTIVE_PHASE=R1"

# 개별 티켓 검증
./docs/agent/verify_ticket.sh W1-3
```

0.6 티켓 스키마 (tickets.yaml)

```
- id: W1-1 # 고유 ID
  phase: R1 # R1 | R2 | R3 | BACKLOG
  title: "..."
  status: ready # backlog|ready|in_progress|review|done|blocked|dropped
  priority: P0 # P0|P1|P2
  effort: M # S(≤2일)|M(3~5일)|L(1~2주)|XL(2주+)
  kind: bind # reuse|bind|new-s|new-l|ops|doc
  depends_on: [W0-2] # 선행 티켓
  human_gate: false # true면 완료 후 사람 승인까지 STOP
  reuse_targets: [경로...] # ⚠ 에이전트가 STEP 2에서 반드시 실제로 읽을 파일
  spec: | # 무엇을 할지 (사람이 관리)
  dod: "..." # 완료 판정 - 사람이 재현 가능한 문장으로
  verify: [명령어...] # STEP 5에서 실행할 검증
  promote_when: "..." # BACKLOG 전용 - 승격 조건
  # 아래는 에이전트가 기록
  blocker: "..." # CLARIFY/BLOCKED 사유·질문
  evidence: "..." # DoD 재현 증거 경로
  updated_at: "..."
```

kind의 의미: `bind` (이미 있는 기능에 데이터만 연결)가 가장 많다는 사실 자체가 이 프로젝트의 성격이다 — 새로 만드는 일보다 연결하는 일이 많다.

W0. 보안 위생 — 선행 필수 [P0]

이 블록이 끝나기 전에는 인증 신청도, 외부 데모도 하지 않는다.

W0-1. 커밋된 시크릿 전량 로테이션 [P0] 공수 S

문제: `backend/.env.example`, `frontend/.env.example`에 실제 운영 키가 그대로 커밋되어 있다.

노출 확인된 항목:

```
frontend/.env.example : VITE_KAKAO_API_KEY, VITE_GOOGLE_MAPS_API_KEY,
                        VITE_TURN_USERNAME, VITE_TURN_PASSWORD, VITE_CGS_APIKEY
backend/.env.example  : DB_PASSWORD, MINIO_ACCESS_KEY, MINIO_SECRET_KEY,
                        SMTP_USERNAME, SMTP_PASSWORD, GCS_APIKEY,
                        OPENSEARCH_USERNAME, OPENSEARCH_PASSWORD,
                        KAKAO_API_KEY, ANYANG_SERVICE_KEY
```

작업 1. 위 전 항목 **실제 값 폐기 후 재발급** (Kakao/Google 콘솔, MinIO, TURN, SMTP 앱 비밀번호, DB, OpenSearch, 안양시 서비스키는 발급기관 문의) 2. 두 `.env.example` 의 값을 **전부 플레이스홀더로 치환** (`your-kakao-api-key-here` 형식) 3. git 히스토리 노출 여부 확인 → 필요 시 `git filter-repo` 또는 최소한 **키 무효화로 대응** 4. CI에 시크릿 스캐너 추가 (`gitleaks` 또는 `trufflehog`) — 이후 커밋 차단

DoD: `.env.example` 두 파일에 실값이 0개. CI에서 시크릿 스캔 잡이 통과.

W0-2. 멀티테넌트 권한 우회 제거 [P0] 공수 M ⚠ 최대 보안 리스크

문제: `backend/common/base_model.py` → `CustomManagerGroup.get_queryset()` 내

```
performance_bypass_models = [
    'order', 'orderitem', 'orderhistory', 'payment', 'ordercomment', 'orderassignment',
    'coreuser', 'usergroup', 'role', 'userprofilelink',
    'multilanguagecontent', 'terminal', 'userprofile', 'group'
]
```

이 목록의 모델은 **group** 격리를 건너뛴다. 단일 기관 온프레미스에서는 무해하나, **SaaS에서는 A 고객이 B 고객의 주문·터미널·사용자 데이터를 조회할 수 있다.**

작업 1. 목록에서 **업무 데이터 모델부터 제거:** `order`, `orderitem`, `orderhistory`, `payment`, `ordercomment`, `orderassignment`, `terminal` - 제거 후 성능 저하가 나면 **인덱스 추가 + `select_related` / `prefetch_related` 로 해결**한다. 우회로 되돌리지 않는다. 2. 프레임워크 모델 (`coreuser`, `role`, `group`, `usergroup`, `userprofile`, `userprofilelink`, `multilanguagecontent`)은 `core` 의존이므로 **즉시 제거 대신 뷰 레벨 필터 강제**: - 사용자 목록/역할 조회 API에 `request.user.group` 기준 명시 필터 추가 3. **테넌트 격리 회귀 테스트 신설** (아래 § W0-3)

DoD: 아래 테스트가 통과하고 CI에 상설화됨.

W0-3. 테넌트 격리 회귀 테스트 [P0] 공수 M 신규

위치: `backend/tests/test_tenant_isolation.py` (신규)

테스트 시나리오 (최소 세트)

```
setUp: group A 사용자 userA, group B 사용자 userB 생성.
      각 group 소유로 Order/Terminal/StreamMonitor/Dashboard/Device/ChecklistSetting 1건씩 생성.

for each model in [Order, Terminal, StreamMonitor, Dashboard, Device,
                   ChecklistSetting, SurveillanceProfile, Handover, ReportTemplate]:
    test_list_api:      userA로 목록 조회 → B의 레코드 id가 응답에 없어야 함
    test_detail_api:    userA로 B 레코드 상세 조회 → 403 또는 404
    test_update_api:    userA로 B 레코드 수정 시도 → 403/404, DB 미변경
    test_delete_api:    userA로 B 레코드 삭제 시도 → 403/404, DB 미변경
    test_export:        userA의 내보내기/리포트에 B 데이터 미포함
```

규칙: 새 모델을 추가할 때 이 테스트 목록에 등록하지 않으면 **머지 금지** (PR 템플릿 체크박스로 강제)

DoD: 위 시나리오 전부 green. CI 필수 잡으로 등록.

W0-4. 데모·목업 코드 분리 [P0] 공수 S

대상

```
frontend/src/features/mockupDemoUi/      (44 파일)
frontend/src/features/setupData/          (/setup-demo-file, /setup-demo-url 라우트)
frontend/src/services/API.ts              → CustomRoutes.setupDemo / setupDemoUrl / djiUrl(임시 라우트 여부)
guardianx-source/drone-monitoring.html    (34KB 루트 데모 파일)
```

작업: 빌드 플래그(VITE_ENABLE_DEMO)로 라우트 등록 자체를 차단. 프로덕션 빌드에서 번들 제외(dynamic import + 조건부 라우트). **이유:** GS인증은 미완성 기능이 UI에 노출되는 것을 결함으로 처리한다. **DoD:** 프로덕션 빌드 결과물에 mockupDemoUi 청크가 없고, 해당 URL 직접 접근 시 404.

W1. 자동 보고서 — 최우선 가치 [P0]

Value 설계서의 담당자 킬러 기능. 이미 있는 템플릿 엔진에 데이터만 연결하는 작업이다. 새 PDF 엔진을 만들지 말 것.

재사용 대상 (반드시 먼저 읽을 것)

```
backend/report_template/                ← 템플릿 CRUD·정적파일·마이그레이션
backend/report_template/management/commands/ ← 기본 템플릿 시드
backend/surveillance/services/video_analysis_report_service.py ← 영상분석 리포트 기존 구현
backend/print_format/                   ← 출력 포맷
backend/fonts/, download_fonts.py, check_fonts.py ← WeasyPrint 한글 폰트
requirements.txt: weasyprint==66.0, django-jinja, liquidjs(FE), nunjucks(FE)
frontend/src/features/reportTemplate/    ← 템플릿 편집 UI (31 파일)
i18n 확인된 기존 템플릿명: "일일/주간/월간 통합관계 업무 보고서"
```

W1-1. 임무 리포트 서비스 [P0] 공수 M 재사용+신규

신규 파일: backend/surveillance/services/mission_report_service.py

인터페이스

```
class MissionReportService:
    @classmethod
    def build_context(cls, mission_id: int) -> dict:
        """리포트 템플릿에 주입할 컨텍스트를 조립한다."""
        # 아래 소스에서 수집:
        # - surveillance.SurveyMission / SurveillanceProfile : 임무 메타(명칭·구역·계획)
        # - flight_log : 실제 비행 경로·시간·고도·거리
        # - stream_monitors.StreamMonitorRecord : 녹화 구간·object_path(MinIO)
        # - (W2 완료 후) events.Event : 탐지 이벤트 목록·스냅샷
        # - devices.Device : 사용 기체·조종자
        return {
            "mission": {...}, "flight": {...}, "events": [...],
            "snapshots": [...], "operator": {...}, "generated_at": ...,
        }

    @classmethod
    def render(cls, mission_id: int, template_code: str = "MISSION_DAILY") -> str:
        """WeasyPrint로 PDF 생성 후 MinIO에 업로드, object_path 반환."""
```

규칙 - PDF 생성은 `report_template` 의 기존 렌더 경로를 그대로 호출한다. WeasyPrint를 직접 새로 호출하지 말 것. - 한글 폰트는 기존 `fonts/` + `check_fonts.py` 검증 경로 사용. - 무거운 작업이므로 **Celery task로 비동기 실행**, 완료 시 `WebSocket(Channels)`으로 알림.

W1-2. 리포트 템플릿 3종 시드 [P0] 공수 S 재사용

`backend/report_template/management/commands/` 에 시드 커맨드 추가: | code | 이름 | 용도 | |---|---|---|
`MISSION_DAILY` | 순찰 임무 보고서 | 임무 1건 종료 시 | | `INCIDENT_REPORT` | 상황 발생 보고서 | 이벤트 확인 처리 시 | |
`MONTHLY_PERFORMANCE` | 월간 운영 성과 보고서 | W3-3 자동 발행용 |

DoD: `python manage.py <seed_cmd>` 실행 후 관리자 화면에서 3종 템플릿이 보이고 미리보기가 렌더된다.

W1-3. API + 화면 [P0] 공수 M

POST <code>/api/surveillance/missions/{id}/report</code>	→ <code>task_id</code> 반환 (비동기)
GET <code>/api/surveillance/missions/{id}/reports</code>	→ 생성된 리포트 목록
GET <code>/api/reports/{report_id}/download</code>	→ MinIO presigned URL

FE: `frontend/src/features/SurveyMission/pages/DetailSurveyMissionPage.tsx` - 상단에 [보고서 생성] 버튼 1개 추가 (`AntD Button type="primary"`) - 생성 중 진행 표시 → 완료 시 다운로드 링크 토스트 - 버튼은 1개다. 옵션 드롭다운·설정 모달을 만들지 말 것. (기본 템플릿 자동 선택)

DoD (담당자가 느끼는 것)

임무 상세 화면에서 [보고서 생성]을 한 번 누르면 1분 이내에 PDF가 생성되고, 그 PDF에 임무명·일시·경로지도·비행시간·탐지 이벤트·스냅샷이 들어 있어 수정 없이 결재에 올릴 수 있다.

W1-4. 인수인계 AI 요약 [P0] 공수 S 재사용

대상: `backend/handover/services/` + `frontend/src/features/Handover/` - 인수인계 생성 시 해당 근무시간대의 이벤트·미처리 건을 자동 조회해 본문에 프리필 - 기존 주간/야간 인수인계 폼 구조를 유지하고 섹션 하나만 추가한다

DoD: 인수인계 작성 화면 진입 시 'AI 요약' 섹션이 이미 채워져 있다.

W2. AI 이벤트 센터 [P0]

현재는 검출 결과가 스트림에 그려질 뿐 이벤트로 축적되지 않는다. 이벤트를 1급 엔티티로 승격한다.

재사용 대상

<code>backend/stream_monitors/models.py</code>	← <code>AIModel</code> , <code>StreamMonitorAIModel</code> , <code>StreamMonitor</code>
<code>backend/stream_monitors/consumers.py</code>	← <code>WebSocket (Channels)</code> 기존 구현
<code>backend/stream_monitors/routing.py</code>	
<code>backend/media_data/services/media_data_detect_service.py</code>	
<code>frontend/src/features/MultiStreamMonitor/</code>	(20 파일)

W2-1. Event 모델 신설 [P0] 공수 M 신규

신규 앱 만들지 말 것. `stream_monitors` 앱 안에 추가한다.

```
# backend/stream_monitors/models.py 에 추가
class DetectionEvent(BaseModelWithGroup): # ← group 격리 필수
    stream_monitor = FK(StreamMonitor)
    ai_model = FK(AIModel, null=True)
    event_type = CharField(...) # person / vehicle / fire / smoke / intrusion / sos
    severity = CharField(...) # info / warning / critical
    occurred_at = DateTimeField(db_index=True)
    lat, lng, alt = FloatField(null=True)
    confidence = FloatField(null=True)
    bbox = JSONField(null=True)
    snapshot_path = CharField(...) # MinIO
    clip_path = CharField(null=True)
    status = CharField(default='new') # new / confirmed / rejected / closed
    reviewed_by = FK(CoreUser, null=True)
    reviewed_at = DateTimeField(null=True)
    reject_reason = CharField(null=True)
    mission = FK('surveillance.SurveyMission', null=True) # 리포트 연결용
```

주의: `BaseModelWithGroup` 상속 필수. W0-3 격리 테스트 목록에 등록할 것.

W2-2. 이벤트 수집 [P0] 공수 M 재사용

`stream_monitors/services/grpc_dual_stream_service.py` 의 검출 결과 수신 지점에서 프레임을 그리는 것과 **별개로** `DetectionEvent` 를 생성한다. - **중복 억제**: 동일 stream+type이 N초(기본 10초) 내 재발생 시 기존 이벤트의 `last_seen` 만 갱신 (알림 폭주 방지 — Value 설계서 "알림 과다" 페인포인트 대응) - 스냅샷 1장을 MinIO에 저장하고 `snapshot_path` 기록

W2-3. 이벤트 센터 화면 [P0] 공수 L 신규

신규: `frontend/src/features/AiEventCenter/` - 3열 레이아웃: **지도 | 라이브 영상 | 이벤트 타임라인** - 이벤트 리스트는 **Antd Table + virtual 가상스크롤 + expandedRowRender** (행 확장으로 스냅샷·상세를 인라인 표시. 별도 모달 금지) - 각 행에 [확인] [기각] **2버튼만** - **정렬 규칙**: 최신순이되 `severity=critical` + `status=new` 는 최상단 고정 - **색 규칙(ISA-101)**: 빨강은 **critical 알람 전용**. 드론 상태·경로 등 다른 요소에 빨강 사용 금지. 배경은 순수 블랙이 아닌 **중명도 그레이**(Carbon Gray 90 계열)

DoD: 이벤트 발생 → 3초 이내 타임라인에 표출 → [기각] 클릭 시 목록에서 사라지고 DB `status='rejected'` .

W2-4. 오탐 피드백 루프 [P1] 공수 S

기각 누적 통계를 `AIModel` 별로 집계해 관리자 화면에 노출. (모델 재학습은 R2)

W3. 3계층 대시보드 프리셋 [P0]

신규 대시보드 엔진을 만들지 말 것. `backend/dashboard/` 에 커스텀 대시보드 빌더가 이미 있다(패널·그룹·가중치 모델 존재). 우리는 **프리셋 3개**를 정의할 뿐이다.

재사용 대상

```
backend/dashboard/models.py, repository/, services/, shemas/  
frontend/src/features/Dashboard/ (93 파일 - 최대 모듈)  
├─ SurveillanceDashboard/  
└─ DeliveryDashboard/ (indexV2, DeliveryDashboardAnyang)  
i18n 확인: "통합 대시보드", "커스텀 대시보드 1~3", "빈 대시보드", "대시보드 정의하기"
```

W3-1. 대시보드 라우트 정리 [P1] 공수 M 정리

현재 6개로 분화되어 있고 **고객사명이 라우트에 하드코딩**되어 있다:

```
/intergrated-dashboard (오타: intergrated → integrated)  
/monitoring-dashboard  
/surveillance-dashboard  
/delivery-dashboard  
/delivery-dashboard-anyang ← 고객사 하드코딩  
/disabillity-dashboard (오타: disabillity → disability)
```

→ `/dashboard/:presetCode` **1개 라우트 + 프리셋 데이터**로 통합. 기존 URL은 301 리다이렉트 유지.

W3-2. 프리셋 3종 정의 [P0] 공수 M 신규(데이터)

preset code	대상	패널 구성
OPERATOR	담당자	오늘의 임무·미처리 이벤트·다음 순찰 일정·최근 보고서
MANAGER	부서장	상단 4카드: 순찰 커버리지 / 탐지 건수 / 평균 응답시간 / 미처리 0건 + 월간 추이 + 구역별 히트맵
EXECUTIVE	기관장	숫자 4~6개 + 지도 1장. 대형 화면·태블릿 대응. 시민 체감 사례 섹션. 스크롤 없이 1화면

로그인 시 사용자 role에 따라 기본 프리셋 자동 분기. (rj-core Role 활용)

DoD (기관장 뷰): 회의실 대형 모니터에서 **스크롤 없이 1분 안에 상황 파악**이 되고, 폰트가 3m 거리에서 읽힌다.

W3-3. 월간 성과 리포트 자동 발행 [P0] 공수 S 재사용

Celery beat: 매월 1일 09:00 → `MONTHLY_PERFORMANCE` 템플릿 렌더 → 부서장 role 사용자에게 메일 발송. **DoD:** 요청 없이 메일함에 PDF가 도착한다.

W3-4. 국비 공모 실적자료 내보내기 [P1] 공수 M 신규

기간 지정 → 비행 횟수·누적 거리·커버 면적·탐지/대응 사례·가동률을 **공모 제안서에 붙여넣을 수 있는 형식**(docx 또는 표 이미지)으로 출력. **DoD:** 담당자가 출력물을 복사→붙여넣기만 하면 공모 제안서 실적 섹션이 완성된다.

W4. 라이선스 미터링 [P0] 공수 M 신규

과금 3축(채널·기체 / 거점 / AI 모듈)을 **제품이 스스로 계측**해야 한다. 현재 계측 없음.

위치: `backend/partner/` 확장 (신규 앱 금지 — Partner 모델이 이미 `api_key`·만료일·콜백을 가진다)

```
class LicensePlan(BaseModel):
    partner/group          # 소유 테넌트
    max_devices, max_channels, max_concurrent_viewers
    enabled_ai_models      # M2M → AIModel
    enabled_modules        # JSON: ['surveillance', 'delivery', 'station', ...]
    tier                   # lite / standard / drone_pro / drone_ultra
    valid_from, valid_to
    offline_key            # 폐쇄망용 서명 키(W4-3)

class UsageSnapshot(BaseModel):
    group, captured_at
    active_devices, active_channels, ai_inference_seconds, storage_bytes
```

W4-1. 계측 [P0] 공수 M

Celery beat 5분 주기로 스냅샷 적재. 스트림 시작/종료 시점에 이벤트 기록.

W4-2. 제한 적용 [P0] 공수 S

초과 시 **차단이 아니라 경고 + 관리자 알림**을 기본으로 한다(공공 고객에게 갑작스러운 차단은 사고로 인식됨). 하드 차단은 계약상 명시된 경우만.

W4-3. 오프라인 라이선스 [P1] 공수 M

인터넷 없이 서명된 키 파일로 활성화/갱신. 폐쇄망 국방·보안기관 요건.

W4-4. 사용량 화면 [P0] 공수 S

관리자 대시보드에 현재 사용량 vs 한도 + CSV 내보내기. **영업 데모에서 그대로 쓰는 화면.**

DoD: 관리자 화면에서 테넌트별 기체 수·채널 수·AI 모듈·저장 용량이 실시간으로 보이고 CSV로 나온다.

W5. VMS 게이트웨이 [P0] 공수 L 신규 — 유일한 완전 신규 핵심 기능

Value 설계서 "새 시스템 학습 부담" 페인포인트의 유일한 해법. **병렬 착수 가능.**

W5-1. ONVIF Profile S 노출 [P0] 공수 L

GuardianX가 **ONVIF 카메라인 척** 드론 채널을 노출한다. - Device Discovery(WS-Discovery), GetProfiles, GetStreamUri, GetSnapshot 최소 구현 - 각 StreamMonitor 를 하나의 ONVIF 프로파일로 매핑 - RTSP 재배포 시 **재인코딩하지 않는다**(패스스루).
이유: A100/H100에 NVENC 없음 → PRD NFR

W5-2. 이벤트 푸시 [P0] 공수 M

DetectionEvent 발생 시 VMS로 전달. 1차는 **이노덱 파트너스 오픈 API**, 2차는 ONVIF Event Service.

W5-3. 연동 검증 [P0] 공수 M

이노덱 VURIX 시험 환경에서 채널 등록 → 라이브 → 이벤트 수신 왕복 확인.

DoD: 이노덱 VURIX 화면에서 GuardianX 드론 채널을 **일반 CCTV처럼 등록·시청**할 수 있고, 탐지 이벤트가 VMS 이벤트 목록에 뜬다.

W6. AI 파이프라인 재설계 [P1→P0 for R2] 공수 XL

R1에서는 인터페이스만 확정하고, 구현은 R2. 지금 구조로도 데모·실증은 되므로 R1 판매를 막지 않는다.

현재 구조의 문제 (실측)

backend/stream_monitors/services/grpc_dual_stream_service.py

```
cv2.VideoCapture(RTSP) → CPU 디코드 (NVDEC 미사용)
batch_size=6, batch_interval=1.0 → 실질 6fps 분석
cv2.imencode('.jpg', frame) → 프레임마다 JPEG 압축 (화질 손실 + CPU)
gRPC 전송 → 렌더링된 프레임 수신 → 네트워크 왕복
cv2.VideoWriter(mp4v) + _start_ffmpeg_stream() → CPU 재인코딩
cap.set(CAP_PROP_FOURCC, 'MJPG') → MJPEG 강제 (대역폭 낭비)
```

결과: 채널당 CPU 코어 1~2개 점유 → 32코어 서버 기준 **동시 16~30채널이 상한**, GPU는 유휴.

W6-0. 사전 실측 (먼저 할 것) [P0] 공수 S

H100 24시간 임대(약 10만원)로 **현재 파이프라인 vs DeepStream 파이프라인의 동시 채널 수를 실측**한다. 실제 드론 영상 3종(주간/야간/열화상)으로 측정. **이 수치 없이 캐파·가격표를 확정하지 말 것.**

W6-1. 목표 구조

```
DeepStream nvuriscbin (NVDEC 디코드, GPU 메모리 상주)
→ 프레임 게이팅 (N프레임당 1회 · ROI) ← 원가 조절 손잡이
→ nvinferserver → Triton (동적 배치, 테넌트별 모델)
→ nvtracker (NvDCF/MV3DT) ← 스킵 프레임 보간
→ 좌표/메타만 WebSocket 전송 (브라우저가 bbox 렌더)
→ 녹화는 remux (재인코딩 없음)
```

W6-2. R1에서 확정할 인터페이스 [P0] 공수 S

아래 계약을 지금 고정하면 W2·W3가 파이프라인 교체와 무관해진다.

```
DetectionEvent 스키마 (W2-1) → 파이프라인이 바뀌어도 동일
WebSocket 메시지 포맷 {stream_id, ts, detections:[{type,bbox,conf,track_id}]}
스트림 시작/중지 API → 백엔드 구현체 교체 가능하도록 추상화
```

규칙: FE는 절대 gRPC 서비스를 직접 호출하지 않는다. 항상 위 3개 계약만 사용한다.

부록 A. 코딩 컨벤션 (이 프로젝트 한정)

항목	규칙
신규 FE 화면	Ant Design 만 사용. MUI/Bootstrap 신규 사용 금지
신규 지도 사용	<code>MapForRouteUnified</code> 래퍼 경유. Kakao/Google 직접 호출 금지
신규 Django 모델	<code>BaseModelWithGroup</code> 상속 필수 + W0-3 격리 테스트 등록 필수
신규 앱 생성	금지 . 기존 45개 앱 중 적합한 곳에 추가한다
상태관리	신규는 Zustand 로 통일 (Redux 신규 사용 금지)
API	OpenAPI 스펙 우선. <code>/api/v1/</code> prefix
비동기 작업	Celery task + Channels 알림. 요청 스레드에서 무거운 작업 금지
커밋	Conventional Commits + 이슈번호
테스트	신규 코드 커버리지 $\geq 70\%$, 테넌트 격리 테스트는 예외 없이 필수
문서	기능 머지 PR에 사용자매뉴얼 갱신 포함 (GS 결함 방지)

부록 B. 디자인 컨벤션 (관제 UI 한정)

- **배경**: 순수 블랙 금지. 지도·영상 타일만 다크, 이벤트/상태 패널은 **중명도 그레이**(Carbon Gray 90). 근거: ISA-101 / ASM Consortium — 다크 배경은 글레어를 유발해 관제실을 어둡게 만들고 교대근무 졸음을 촉발
- **색**: 빨강 = **critical 알람 전용**. 다른 용도 사용 금지("색은 배타적으로 코딩")
- **알림**: 색 단독 금지. 항상 **색 + 아이콘 + 텍스트** 3중
- **폰트**: 관제 화면 본문 최소 14px (상황실 대형 모니터 기준)
- **파괴적 동작**(임무 중단·복귀): 2단 확인 + 되돌리기 불가 명시
- **키보드**: 전 기능 단축키 매핑(관제사 마우스 의존 제거)
- **컴포넌트**: Storybook 단일 소스. 화면별 임의 스타일 금지

부록 C. Phase Exit 체크리스트

에이전트는 `phase-complete` 정지 시 이 목록을 자동 출력한다. 승격은 사람이 결정한다.

C-1. R1 "Certify" Exit (2026.12 코드 동결)

- [] W0-1 .env.example 실값 0개 + CI 시크릿 스캔 통과
- [] W0-2 performance_bypass_models에서 업무 모델 전부 제거
- [] W0-3 테넌트 격리 테스트 9개 모델 × 5시나리오 green, CI 필수 잡
- [] W0-4 프로덕션 빌드에 데모 청크 없음
- [] W1 임무 상세에서 [보고서 생성] 1클릭 → 1분 내 PDF (수정 없이 결재 가능 품질)
- [] W1-4 인수인계 화면 진입 시 AI 요약 프리필
- [] W2 이벤트 발생 → 3초 내 타임라인 표출, [확인]/[기각] 동작
- [] W3 OPERATOR/MANAGER/EXECUTIVE 프리셋 3종, role별 자동 분기
- [] W3-3 월간 성과 리포트 자동 메일 발송 1회 이상 성공
- [] W4 테넌트별 사용량(기체·채널·모듈·용량) 실시간 표시 + CSV
- [] W5 이노덱 VURIX에서 드론 채널 등록·시청·이벤트 수신 시연 성공
- [] W6-0 H100 실측 완료, 캐파 수치 확정
- [] KPI 편리성 (교육 4h·5클릭) / 완결성 (2시나리오 100%) / 안정성 (가동률 99.5%·72h 무장애) / 참신성 (단독기능 2개)
- [] GS 사용자매뉴얼 전 문장이 실제 동작과 일치 (내부 교차시험 1회전 완료)

C-2. R2 "Station" Exit (2027.2Q)

- [] 동시 채널 수가 W6-0 실측 대비 3배 이상 (W6-1)
- [] Dock 3 상태 수신·명령 왕복 지연 < 1s (W7-1)
- [] 동일 미션이 DJI Dock·국산 스테이션 양쪽에서 실행 (W7-2)
- [] 이륙~복귀~충전 무개입 100회 연속 성공 (W8-1)
- [] 7일 무인 연속 운영 결측 0건 (W8-2)
- [] SAHI ON/OFF 검출률·채널 수 측정 문서화 (W9-1)
- [] 외부망 차단 환경에서 전 P0 기능 동작 (W10-1)
- [] 통신 목적지 리포트로 DJI 서버 미경유 증명 (W10-2)
- [] 디지털서비스몰 등록 완료 (W11-1)
- [] MIG 슬라이스가 K8s 노드 리소스로 광고 (W11-2)

C-3. R3 "Mission" Exit (2027.4Q)

- [] 외부 시스템에서 미션 트리거(DaaS API) 데모 성공 (W12-1)
- [] 영상 수정 시 증거체인 검증 실패가 재현 (W14-1)
- [] 증적 zip 30초 내 생성 + 위변조 검증 통과 (W14-2)
- [] 반출 워크플로에서 개인정보 마스킹 적용 (W14-3)
- [] 파일럿 앱 베타 + 데이터 수집 동의 플로우 동작 (W16-1)

C-4. 백로그 승격 판정 (매 Phase Exit 시 재검토)

id	승격 조건	판정
B-07 AI 모델·데이터 자산화	즉시 — 경영 실사	☐
B-08 상표 출원	즉시	☐
B-05 배송 라이선스 분리	W4-1 완료 후 즉시	☐
B-03 지도 3종 통합	W10-1 착수 직전 또는 지도 버그 월 3건+	☐
B-04 UI 라이브러리 통일	디자인 컨벤션 확정 + R1 출시 후	☐
B-01 안티드론	국방·공항·발전소 고객 요구 확인	☐
B-02 익명 벤치마크	유료 고객 3곳 + 전원 데이터 활용 동의	☐
B-06 다국어 확장	해외 파트너 계약 체결	☐

부록 D. AI 코딩 에이전트용 실행 규칙

1. **작업 시작 전 반드시** 해당 티켓의 "재사용 대상" 파일을 먼저 읽는다. 읽지 않고 새로 만들면 중복 구현이 된다.
2. **한 번에 한 티켓**. W 번호 하나를 DoD까지 끝내고 다음으로 간다.
3. **금지 구역(§ 0.4)에 속한 파일을 수정해야 한다면 작업을 멈추고 보고한다**. 임의 판단 금지.
4. 새 Django 모델을 만들 때는 `BaseModelWithGroup` 상속과 격리 테스트 등록을 **같은 커밋에** 포함한다.
5. DoD를 재현할 수 없으면 **완료로 보고하지 않는다**. 무엇이 막혔는지 구체적으로 보고한다.
6. 성능 문제를 만나면 **권한 필터를 우회하는 방식으로 해결하지 않는다**(W0-2가 그 실수의 결과다). 인덱스·캐시·쿼리 최적화로 해결한다.

부록 E. 티켓 통계 (tickets.yaml 기준)

구분	값
총 티켓	58개 (v2.0의 54 + W0-0/W0-1b/W0-5/W0-6)
Phase 분포	R1 28 · R2 14 · R3 8 · BACKLOG 8
의존성 무결성	깨진 참조 0건 (검증 완료)
<code>gate: human-required</code>	9개 — 에이전트가 STOP
<code>gate: evidence-review</code>	49개 — 증거 제출로 자동 통과
사전 결정	20건 (<code>decisions.yaml</code>)
정본 SHA256	<code>tickets.sha256</code> 참조 — 루프 STEP 0에서 대조
kind 분포	<code>bind</code> · <code>reuse</code> 가 다수 — 연결 작업이 신규 개발보다 많다

부록 F. 자주 나올 CLARIFY 상황과 대응 (에이전트 참고)

상황	에이전트가 할 일	하면 안 되는 일
티켓이 전제한 모델/함수가 코드에 없음	실제 구조를 보고하고 A안/B안 제시 후 STOP	비슷한 걸 알아서 만들기
DoD의 수치 기준이 애매 (예: "빠르게")	구체 수치를 제안하고 확인 요청	임의 기준으로 통과 처리
기존 구현이 티켓과 다른 방식으로 이미 존재	중복 구현 위험 을 보고하고 통합/대체 여부 질의	나란히 하나 더 만들기
성능 테스트가 느려서 CI가 오래 걸림	최적화 방안 제시	테스트 스킵·비활성화
마이그레이션이 기존 컬럼을 삭제해야 함	영향 행 수 보고 후 STOP(<code>destructive-migration</code>)	그냥 실행
금지구역 파일을 고쳐야 기능이 완성됨	저축 사유와 대안 보고 후 STOP	조용히 수정
티켓 spec보다 더 좋은 설계가 보임	제안을 보고서에 적고 현 스펙대로 구현	임의로 더 좋게 구현

부록 G. 사전 결정 레지스트리 요약 (decisions.yaml)

전문은 `agent/decisions.yaml`. 에이전트는 CLARIFY 직전에 이 파일을 조회한다.

G-1. 2026-08-13 W0-1 STOP 해소 결정 (7건)

id	결정	영향
D-001	CI = GitHub Actions 파일 선제 배치 + 로컬 pre-commit이 실패 차단선. DoD를 "CI 잡 green"에서 "로컬 혹은 차단 재현 + CI 파일 존재"로 재정의	W0-1a
D-002	실키 백업은 저장소 밖 암호화(<code>~/.guardianx-secrets/*.7z</code>). 저장소 내부 금지	W0-0, W0-1b
D-003	<code>GCS_APIKEY</code> ≠ <code>VITE_CGS_APIKEY</code> 분리 발급 → 클라 키 도메인 제한 → W0-5로 proxy 전환 후 클라 키 삭제	W0-1b, W0-5
D-004	W0-0 신설: .gitignore 정비 → 플레이스홀더 치환 → 그 다음에 최초 커밋. 실키가 히스토리에 안 들어감	전 티켓 순서
D-005	W0-0의 최초 커밋 = baseline. 이후 변경 파일만 신규 컨벤션 적용. baseline 없으면 SKIP	컨벤션 게이트
D-006	npm 확정 (yarn.lock 86B 빈 파일 → 삭제)	W0-0, verify
D-007	백엔드 검증은 컨테이너에서. 환경 미가용은 FAIL이 아니라 <code>env-unavailable</code>	전 백엔드 티켓

G-2. 루프 주행 규칙 결정 (5건)

id	결정
D-008	티켓당 브랜치 <code>w/<ID>-<slug></code> , squash merge, 에이전트는 머지하지 않는다
D-009	파괴적 마이그레이션 금지 — 컬럼 삭제는 1릴리스 지연 2단계
D-010	<code>human_gate</code> → <code>human-required</code> (9) / <code>evidence-review</code> (49) 이원화. 정지 지점 22→9
D-011	Phase 자동 승격 — P0 전부 done + 미완이 human-required뿐이면 에이전트가 넘어가고 보고
D-012	tickets.yaml 재구성 금지 — 정본 1개 + manifest + SHA256으로 무결성 검사

G-3. 선제 결정 — 나올 것이 확실한 질문 (8건)

id	상황	결정 요지
D-101	기존 구현과 티켓 충돌	새로 만들지 않는다. 확장하거나 어댑터. 나란히 하나 더 만들기 금지
D-102	DoD 수치가 모호	기본값 적용(화면 p95 500ms / API 300ms / 비동기 1분 / 실시간 3초 / 가동률 99.5%). CLARIFY 하지 않음
D-103	성능 저하 발생	인덱스→prefetch→분할→캐시 순. 권한 우회는 절대 금지
D-104	외부 AI 서비스 변경 필요	이 저장소에서 수정 금지. 어댑터 + 변경요청서
D-105	격리 테스트가 기존 기능을 깨뜨림	테스트가 아니라 코드를 고친다. 남의 데이터 의존이 버그
D-106	i18n 문자열 추가	ko/en/th 3종 동시 (태국어 배포 보호)
D-107	신규 라우트 네이밍	kebab-case 정자법. 기존 오타(intergrated/disability) 따라가지 않음
D-108	커밋 단위	티켓당 1커밋. 모델+격리테스트 / 기능+매뉴얼 / 화면+i18n은 같은 커밋

부록 H. 최종 Phase 완주 실행 순서 (요약 카드)

```
# — 0. 배치 (1회)
cp -r agent/ <repo>/docs/agent/
sha256sum -c docs/agent/tickets.sha256      # 정본 무결성 확인

# — 1. 사람이 먼저 할 일 (W0-0의 선행 조건)
# 실키 15개를 저장소 밖에 암호화 백업 (D-002)
# → 백업 완료로 에이전트에게 알린다. 이 확인 없이 치환하면 안 된다.

# — 2. 루프 투입
claude "docs/agent/AGENT_LOOP.md 를 읽고 그대로 실행하라.
ACTIVE_PHASE 는 tickets.yaml 을 따르고, AUTO_PROMOTE=true 로 R3까지 진행하라.
실키 백업은 완료되었다."
```

```
# — 3. 에이전트가 STOP 할 때 (총 9개 지점)
# awaiting-human-review → 해당 티켓의 물리 작업 수행 후 재투입
# clarify                → 제안된 D-XXX 초안을 검토하고 decisions.yaml 에 추가 후 재투입
# env-unavailable        → docker compose up -d 후 재투입

# — 4. 개별 검증
./docs/agent/verify_ticket.sh W1-3
```

예상 주행 거리

구간	정지 없이 갈 수 있는 티켓	정지 사유
W0-0 → W0-1a	2개	W0-1b(사람: 실키 재발급)
W0-2 → W3-4	약 15개	W6-0(사람: H100 임대)
W4-1 → W5-2	약 6개	W5-3(사람: 이노덱 연동)
R2 자동 승격 → W10-2	약 10개	W11-1(사람: CSAP)
R3 → 완주	약 7개	W13-1·W16-1(사람: 사업 판단)

즉 사람은 9번만 개입하면 R3까지 도달한다. v2.0의 22번에서 절반 이하로 줄었다.